

Assignment : Image Segmentation

Signal and Image Processing

February 25, 2026

You can work on this assignment and submit your solution (report and code) as a GROUP. This assignment counts towards your grade and have to be submitted in order to pass the course. You must follow the report guidelines found in `guidelines.pdf`. For test images, scripts and data files - see `Files/Material/Test Images/Week 7` on Absalon. The page limit for this assignment is **15 pages** including everything, i.e. illustrations and code snippets.

1. Segmentation

- 1.1. (1 point) Consider the two images `matrikelnumre_nat.png` and `matrikelnumre_art.png`. There is a clear distinction between foreground (map) and background (desk) in both. Your task is to perform foreground-background segmentation on these images. You can use your solution to exercise 2.1 in the week 4 individual assignment as your starting point. Implement thresholding based segmentation using Otsu thresholding applied to a gray scale version of the images. You are allowed to use `skimage.filters.threshold_otsu`.

Deliverables: Include essential code snippets. Plot the two images in both color and gray-scale. Plot the intensity histogram of the gray-scale versions and explain why we can do foreground-background segmentation on these images. Plot the results of segmenting the two images and compare and explain the results for the two images.

- 1.2. (1 point) Use the canny edge detector to perform edge-based segmentation on `matrikelnumre_art.png`.

Deliverables: Include a code snippet. Plot the result of edge-based segmentation. How does edge-based segmentation compare to threshold-based segmentation? Did it perform better or worse? What are the advantages and disadvantages of edge-based segmentation compared to thresholding-based segmentation?

2. Image Transform

- 2.1. (1 point) Finding the four corners of the map in `matrikelnumre_nat.png` is crucial for the next part of the exercise. You can use your solution to Harris corner detection from week 5 as your starting point. Notice that the map contains a lot of straight lines that connect at right angles, hence, creating a multitude of corners. For your own sake, you should detect as few corners as possible, but the map papers corners must be detected. Think outside of the box and manipulate the image, such that the Harris corner detection algorithm finds as little corners on the map itself.

Deliverables: Provide essential code snippets. Plot the original `matrikelnumre_nat.png` with the detected corners marked on it as dots with a color of your choice. Explain your approach to reducing the number of unimportant corners detected. Report the cartesian coordinates of the map's four corners.

- 2.2. (1 point) As you can see, the map itself is slightly tilted and the image is captured at an angle. This makes the map inconvenient to work with in further processing. Your task is to transform the map to bird's eye view such that the map borders fill the complete resulting image (i.e. the desk background pixels are not visible). This requires a homography transform that you can estimate based on the map corners you detected in the previous question. You are allowed to use functions from the `skimage.transform` package for this purpose.

Deliverables: Provide essential code snippets. Define the 2D homography transform mathematically. Plot the transformed map as an image that has the same dimensionality as the original image. It is important that the transformed map preserves all information present in the original, i.e. there are no streets or land areas lost due to cropping.

3. The Hough Transform

- 3.1. (1 point) Implement a straight line Hough Transform - do not use a library function that computes the Hough transform such as the `hough_line` method in `scikit-image`.

Deliverables: Include essential code snippets illustrating your implementation. Describe how your implementation works and possible issues your implementation can have (e.g. computational complexity).

- 3.2. (1 point) Test your implementation on the image `cross.png`.

Deliverables: Plot both the Hough transform of the image and the detected lines overlayed on `cross.png`. Compare the output of your implementation to the output of the `hough_line` method in `scikit-image`.

- 3.3. (1 point) We now return to the analysis of the map image `matrikelnumre_nat.png` and the transformed image from question 2.2.. Your task is to detect the large blue circle on the map and find the pixel corresponding to its centre. Use the `hough_circle` method of `scikit-image` to detect the blue dot center.

Deliverables: Include essential code snippets. Plot the bird's eye view transformed map and draw a red dot at the center of the blue circle as well as a red circle around the blue circle. Report the cartesian coordinates of the circle centre. Explain how the circle Hough transform works.

4. Morphology

- 4.1. Apply, separately, morphological 'opening' and 'closing' with a disk SE (e.g. using `disk`, radius 1), to the input image `cells_binary_inv.png` (available on Absalon). Using function found in `skimage.morphology` is allowed.

- 4.1.1. (1 point) **Deliverables:** For each of the two operations, show the resultant image along with the original. Select a region and show a zoom-in of this region for the original and the two result images. Highlight a few key locations and describe the changes.

- 4.1.2. (1 point) **Deliverables:** Answer these questions:

A. Why are the two operations not producing the same result?

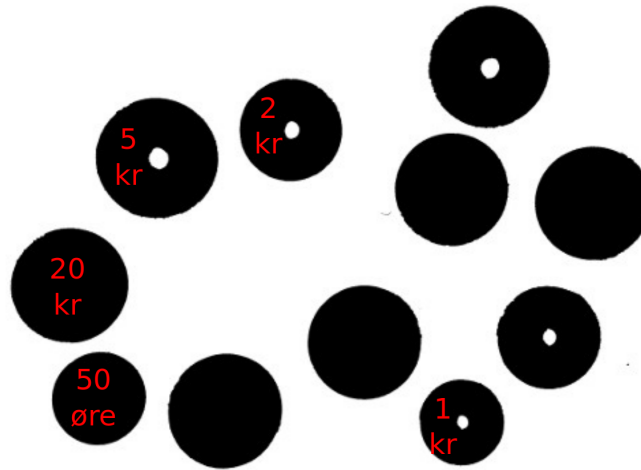


Figure 1: Given the underlying binary image `money_bin.jpg` and the labels seen in this figure, can you write a program that counts the money? For the non-Dane, 100 Øre = 1 kr.

- B. What are the challenges of using just these two operations for separating this particular image into individual cells?
- 4.1.3. (1 point) Use connected components to label the individual cells in both the results of performing opening and closing from question 4.1.1. Use 8-connectivity to define neighbors. We suggest that you use the implementation of connected components in `skimage.measure.label`, but notice that the documentation calls 8-connectivity for 2-connectivity.
- Deliverables:** Include a code snippet illustrating how you compute the connected components. Provide an illustration of the result of labelling the cells using the connected component algorithm for both the opening and closing results. Report how many different connected components (labels) you get. Reflect on how well you have succeeded in identifying the individual cells in the image using either opening and closing.
- 4.2. (1 point) Using mathematical morphology, can you count how much money there is in the image `money_bin.png` on Absalon. You are also given the additional information of the labeled image found in figure 1.
- Deliverables:** Please include both a description of what you did, and the final amount.
- 4.3. (1 point) The binary segmentation image you produced in question 1.1. for the image `matrikelnumre_nat.png` contains black pixels representing building and road lines in the map. Using binary morphology, write a program to remove these lines, but without destroying the shape of the map outline and the round shape coming from the blue dot on the map. You may use the `binary` functions from `skimage.morphology`.
- Deliverables:** Argue for your choice of morphological operation and structuring element. Provide the essential code snippet in the report. Illustrate the result of your solution by visualising the cleaned binary segmentation mask.